

实验 7 异常和中断处理设计与实现

在程序的执行过程当中, 有时会出现预想不到的情况, 如计算结果的溢出、用于地址转换的 TLB 不命中等。另外, 外部设备也可能向 CPU 发出信号, 要求 CPU 为它提供服务。前者称为异常, 后者称为中断。本实验介绍 CPU 如何处理异常和中断, 并给出相应的 Verilog HDL 代码。

1 异常和中断

本节讨论异常和中断的基本概念以及处理异常和中断的基本方法。

1.1 异常和中断的定义与类型

异常 (Exception) 和中断 (Interrupt) 是两种不可预知的事件。异常来自于 CPU 的内部, 如做除法时除数为 0; 而中断来自于 CPU 的外部, 如键盘中断。因此, 如果分别称异常和中断为“内部同步异常”和“外部异步中断”, 则更能反映出它们的准确含义。

当一个异常或一个中断出现时, CPU 应该停止当前程序的执行、转去执行预先准备好的程序去处理这个异常或中断。这个过程称为异常或中断响应。被执行的程序称为异常或中断处理程序。图 1 示出了处理流程。

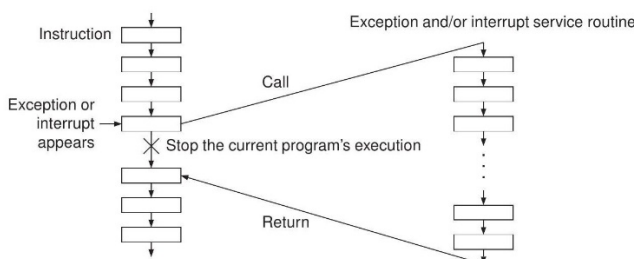


图 1 异常或中断的响应过程

溢出是一种常见的异常。溢出是指计算结果太大或太小而无法正确表示。例如 32 位补码表示的数据可以表示的数的范围是 $-2^{31} \sim +2^{31} - 1$ 。如果计算结果大于 $+2^{31} - 1$ 或小于 -2^{31} , 则出现溢出。还有一种可能的情况就是取来的指令不知道是什么意思, 即指令操作码出错; 或者取来的指令是一条意义明确的指令, 但硬件没有实现它, 而是通过调用子程序来实现它的功能; 或者用户程序试图执行一条只有操作系统才有权利执行的特权指令。其他的异常包括存储器地址没对准、存储器页失效 (CPU 要访问的内容不在存储器中, 这是虚拟存储器管理中经常出现的正常情况)、存储器违法访问 (用户程序往操作系统区域写数据)、断点和单步执行 (为方便程序调试而设置) 等。

系统调用是一条指令, 用户程序通过它来享受操作系统提供的服务。对系统调用指令的处理与对通常的异常和中断的处理非常类似, 因此我们也把它算作异常。

外部中断也有很多种情况, 比如键盘中断、鼠标中断、打印机中断、计时器 (Timer) 的中断和硬件故障等。这些中断的共同特点是 I/O 设备要求 CPU 关照它们一下。

1.2 查询中断和向量中断

当异常或中断发生时, CPU 跳转到一个固定的地址, 从那里开始查询到底出了什么状况, 再转去执行相应的异常或中断处理程序。那么 CPU 如何知道已经发生的是什么异常或中断并如何做出相应的处理呢? 可以设置一个专门的寄存器: 当有异常或中断发生时, 硬件能把发生源的信息记录到这个寄存器。CPU 可以在入口处读取这个寄存器, 然后转移到专门的程序。MIPS 称这个寄存器为 Cause 寄存器, 见图 2。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					IP				0							0	0

图 2 MIPS Cause 寄存器 (CPO 寄存器 13, 只列出了与异常或中断有关的位)

Cause 寄存器中第 6 到第 2 位的 ExcCode 是引起异常或中断事件的代码、IP[7:0] 指出有哪些中断在等着处理。ExcCode 的定义见表 1。

表 1 MIPS Cause 寄存器中 ExcCode 的定义

ExcCode 值	助记符	种类	描述
0	Int	中断	中断 (IP[7:0] 指出中断源)
1	Mod	异常	TLB 项匹配但存储器页还没被写过 (存数据时)
2	TLBL	异常	TLB 项不匹配或无效 (取数据或取指令时)
3	TLBS	异常	TLB 项不匹配或无效 (存数据时)
4	AdEL	异常	存储器地址错 (取数据或取指令时)
5	AdES	异常	存储器地址错 (存数据时)
7	IBE	异常	总线错 (取指令时)
8	DBE	异常	总线错 (取数据或存数据时)
9	Sys	异常	执行系统调用指令
10	Bp	异常	执行断点指令
11	CpU	异常	试图执行保留的指令
13	Ov	异常	算术操作时结果溢出
14	Tr	异常	执行陷阱指令
15	FPE	异常	浮点操作结果不正确
16 ~ 22	-	-	保留
23	WATCH	异常	虚拟地址与 Watch 寄存器的内容一致了
24	MCheck	异常	TLB 项匹配了不止一个, 但不一致
25 ~ 29	-	-	保留
30	CacheErr	异常	Cache 出错
31	-	-	保留

3. 中断返回

CPU 在处理完异常或中断后, 返回到当初被打断的程序继续执行。在转向异常或中断处理程序时, 不要忘记把返回地址保存到一个安全的地方。有些 CPU 的作法是把返回地址保存到一个通用寄存器中; 有些 CPU 的作法是设置一个专门的寄存器, 用于保存返回地址, 比如 MIPS CPU 中的 EPC 寄存器; 有些 CPU 则是把返回地址保存到存储器堆栈那么栈顶指针应该能够自动调节, Intel x86 CPU 采用的就是这种办法。

MIPS CPU 使用 `eret` 指令从异常或中断处理程序返回: 把 EPC 的值写回 PC。因此, 当某些引起异常的指令不需要重新执行时, 返回之前要把 EPC 加 4, 见下面的代码。当然, 如果需要重新执行, 就不用加 4 了。外部中断发生时, 返回地址是下一条指令的地址 (NPC), 因为外部中断发生时, 当前 PC 所指的指令要执行完毕后, 才转去执行异常或中断处理程序。

```
mfc0 $k0, C0_EPC    # 把 EPC 寄存器的内容送到寄存器 r26
addiu $k0, $k0, 4    # 加 4
mtc0 $k0, C0_EPC    # 送回 EPC 寄存器
eret                # 从异常处理程序返回: PC <-- EPC
```

1.3 中断屏蔽和中断嵌套

执行中断处理程序时又出现中断请求怎么办? 通常的做法是进入中断处理程序时就自动屏蔽中断, 即“关中断”: 来了中断 CPU 也不理睬。如果 CPU 想理睬呢? 这就需要“开中断”。图 3 示出了 MIPS CPU Status 寄存器中有关中断屏蔽的控制位。IM[7:0] 中的每一位控制一个中断: 为 0 时禁止中断, 为 1 时允许中断。IE 是 IM[7:0] 的使能信号。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				IM					0	0	0	0	0	0	0	IE

图 3 MIPS Status 寄存器 (CPO 寄存器 12, 只列出了与中断屏蔽有关的位)

在处理当前的中断时又去响应新的中断, 称其为中断嵌套 (Interrupt Nesting), 如图 4 所示。只有开了中断才会嵌套。开中断前要把“现场”保存好, 尤其是专用寄存器或通用寄存器中的返回地址。

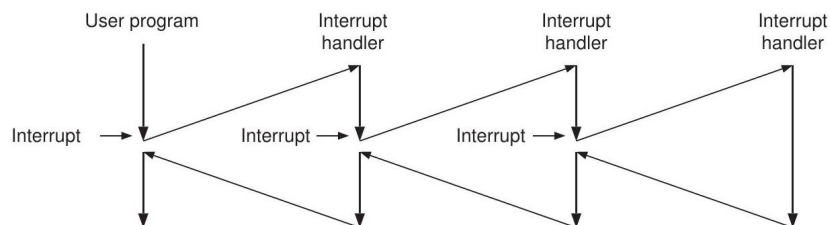


图 4 中断嵌套

2 带有异常和中断处理功能的 CPU 的设计

本节具体设计带有异常和中断处理功能的简化 CPU。本实验做如下假定:

1. 只有一个外部中断请求;
2. 只处理 3 个异常: 结果溢出、出现未实现的指令以及系统调用;
3. 返回地址: 异常时保存当前指令的地址, 系统调用和中断时保存下一条指令的地址;
4. 采用查询中断方式;
5. 响应异常或中断时把 **Status** 寄存器的内容左移 4 位自动关中断;
6. 中断返回时把 **Status** 寄存器的内容右移 4 位 (恢复原来的内容)。

2.1 异常和中断的处理过程以及相关的寄存器

CPU 响应一个内部同步异常或外部异步中断时, 有以下 4 件事情要同时做。

1. 把返回地址保存到 **EPC** 寄存器中, 以便处理完异常或中断后返回;
2. 把引起异常或中断的原因自动记录到 **Cause** 寄存器 (**ExcCode** 域);
3. 把 **Status** 寄存器中的中断屏蔽位左移 4 位 (自动关中断);
4. 把一个固定的地址 (异常和中断处理程序的入口地址) 写入 **PC**。

如果是中断请求 **intr**, CPU 还要发出中断确认 **inta**。异常或中断处理程序可以读取 **Cause** 寄存器的内容来确认到底发生了什么事情, 依此再跳转到相应的程序去对异常或中断进行处理。

处理完毕后, CPU 要返回到原来被打断的地方继续执行。通过执行 **eret** 指令实现。该指令要完成的动作有两个: 把 **EPC** 的内容写回到 **PC**; 与此同时, 把 **Status** 寄存器中的内容右移 4 位 (恢复原来的中断屏蔽设置)。

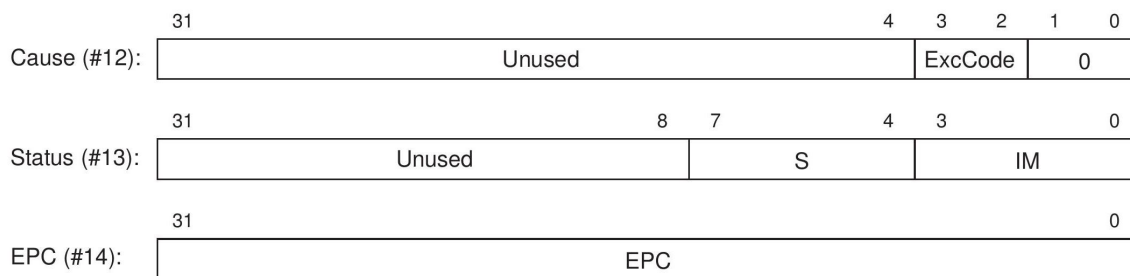


图 5 与异常和中断处理有关的 3 个寄存器

图 5 示出的是上面提到的 3 个寄存器。它们都属于 CPO 寄存器, 每个寄存器都有一个号码, 以便 CPU 使用 **mfc0** 或 **mtc0** 指令访问它。引起异常或中断的原因在 **Cause** 寄存器的 **ExcCode** 中, 其编码见表 2。**Status** 寄存器中的 **IM[3:0]** 是 4 位屏蔽位, 每位对应一个异常或中断。屏蔽位为 1 时允许异常或中断、为 0 时禁止。**S[3:0]** 是左移 4 位的 **IM[3:0]**。**EPC** 用于保存返回地址。

表 2 ExcCode 及 IM[3:0] 的定义

ExcCode 值	助记符	种类	屏蔽	描述
0	Int	中断	IM[0]	外部中断
1	Sys	异常	IM[1]	执行系统调用指令 (多正常啊)
2	Unimpl	异常	IM[2]	试图执行没有实现的指令
3	Ov	异常	IM[3]	算术操作时结果溢出

2.2 与异常和中断有关的指令

引起算术计算结果溢出的指令有 3 条: add、sub 和 addi。这 3 条指令都是对带符号数进行计算。实际上对 ALU 来讲只有加减运算, 因为 ALU 并不区分是 add 还是 addi。MIPS 还有对无符号数进行计算的指令, 比如 addu。即使结果溢出, 这类指令也不产生异常。表 3 列出了在什么情况下溢出 ($v = 1$), 其中 $a[31]$ 和 $b[31]$ 分别是补码表示的 32 位操作数 a 和 b 的最高位; $r[31]$ 是计算结果的最高位。

表 3 加减操作的溢出

操作	aluc[3:0]	a[31]	b[31]	r[31]	v	解释
ADD	x000	0	0	1	1	正数加正数结果反而为负数
ADD	x000	1	1	0	1	负数加负数结果反而为正数
SUB	x100	0	1	1	1	正数减负数结果反而为负数
SUB	x100	1	0	0	1	负数减正数结果反而为正数

根据表 3, 直接写出溢出信号 v 的逻辑表达式 (Verilog HDL 格式):

```
v = ~aluc[2] & ~a[31] & ~b[31] & r[31] & ~aluc[1] & ~aluc[0] |
    ~aluc[2] & a[31] & b[31] & ~r[31] & ~aluc[1] & ~aluc[0] |
    aluc[2] & ~a[31] & b[31] & r[31] & ~aluc[1] & ~aluc[0] |
    aluc[2] & a[31] & ~b[31] & ~r[31] & ~aluc[1] & ~aluc[0];
```

CPU 可以使用 mfco rt,rd (Move from C0) 和 mtco rt,rd (Move to C0) 指令读写寄存器 Cause、Status 或 EPC。这两条指令中的 rt 是通用寄存器的号码, rd 是寄存器 Cause、

Status 或 EPC 的号码。我们还需要有系统调用指令 `syscall` 和从异常或中断返回的指令 `eret`。以上 4 条指令的格式见图 6。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
mfc0	0	1	0	0	0	0	0	0	0	0	0			rt					rd				0	0	0	0	0	0	0	0	0	0	
mtc0	0	1	0	0	0	0	0	0	1	0	0			rt					rd				0	0	0	0	0	0	0	0	0	0	
syscall	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
eret	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0

图 6 mfc0、mtc0、syscall 和 eret 指令的格式

2.3 带有异常和中断处理功能的 CPU 总体结构

经过以上的讨论, 已经清楚了处理异常或中断的机制。一种带有异常和中断处理功能的单周期 CPU 的总体模块如图 7 所示。

图 7 中新加了 3 个 CPO 寄存器 (Status、Cause 和 EPC) 以及对它们进行读写所需要的模块 (主要是多路选择器)。另外新加的模块是 PC 值的选择。除原来的之外, 还应实现向异常或中断处理程序的跳转及返回。所有新加的模块所需的控制信号要由控制部件产生。

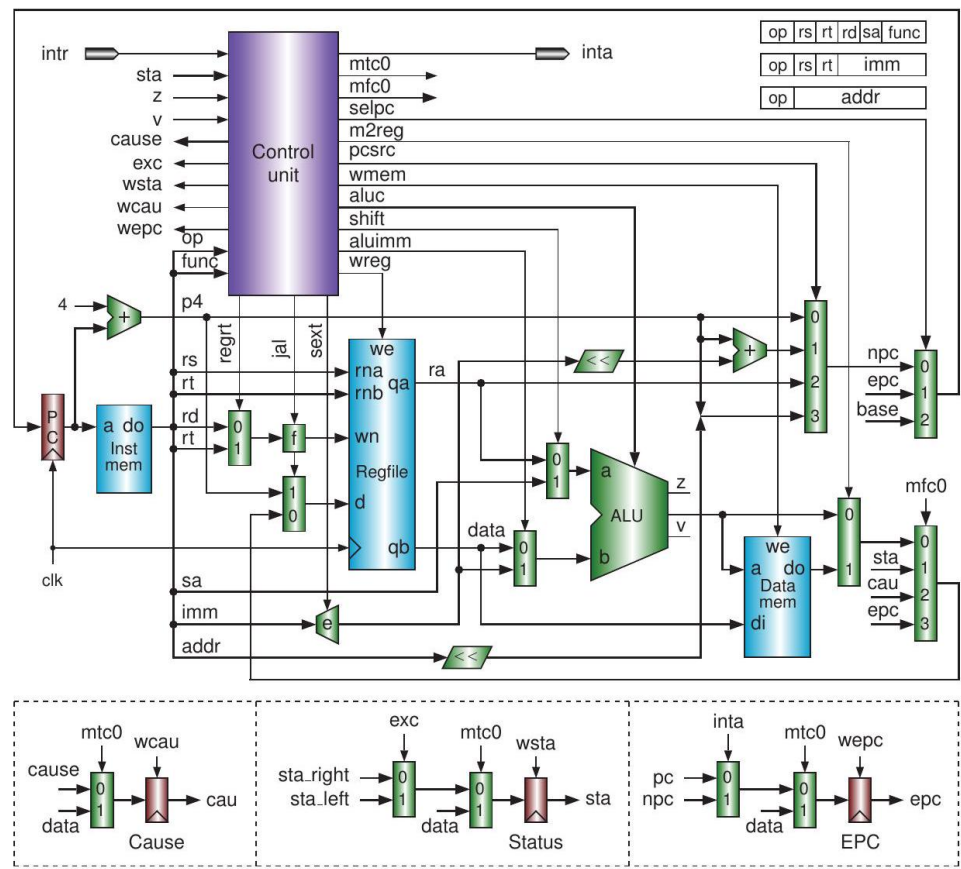


图 7 带有处理异常或中断功能的单周期 CPU

现把模块的工作过程说明如下。先说 `mfc0` 和 `mtc0` 两条指令的执行状况。`mfc0` 指令把 `Status`、`Cause` 或者 `EPC` 寄存器中的数据写入通用寄存器堆。寄存器数据的选择由图中最右边的四选一多路器完成, 选择信号为 `mfc0`。`mtc0` 指令把通用寄存器堆中的一个寄存器的内容写入 `Status`、`Cause` 或者 `EPC` 寄存器。相应的写使能信号分别为 `wsta`、`wcau` 和 `wepc`。三个寄存器左边的二选一多路器的选择信号均为 `mtc0`。这时的 `mtc0` 应为 1, 选择寄存器堆的 `qb` 的输出, 即图中命名为 `data` 的数据。

如果是外部中断, 输入管脚 `intr` 为 1; 如果是溢出, `ALU` 的输出信号 `v` 为 1, 送到控制部件; 通过对当前指令译码, 可知是否为未实现的指令或者 `syscall` 指令。总之控制部件知道是否出现了异常或中断。这时要使用 `Status` 寄存器的信息 `sta` 来查看相应的异常或中断是否被屏蔽。如果异常或中断出现了且没被禁止, 控制部件生成 `ExcCode`, 经由 `cause` 信号送给 `Cause` 寄存器左边的多路器。这时多路器的选择信号 `mtc0` 应为 0, 以把 `cause` 数据写入 `Cause` 寄存器。在执行 `mtc0` 指令前关掉所有的中断和异常。其他两个寄存器在异常或中断发生时也有数据要写入。往 `EPC` 中写入的是返回地址。中断时返回地址是下一条指令的地址, 即图中的 `npc`; 异常时返回地址是引起异常的指令的地址, 即图中的 `pc`。选 `pc` 还是 `npc` 由 `inta` 决定。`inta` 是 CPU 的输出信号, 意义为中断确认 (或中断回答或中断应答等, 总之是英文 `Interrupt Acknowledgement` 的意思)。往 `Status` 寄存器写入的是 `Status` 寄存器左移 4 位后的内容, 即图中的 `sta_left`。由于 `Status` 寄存器的最低 4 位是异常或中断屏蔽位, 左移 4 位再写入相当于既保存了原来的屏蔽信息又禁止了异常或中断 (左移后最低 4 位填 0)。因此, 左下角的多路器的选择信号 `exc` 应为 1, 选择 `sta_left`。当然还要修改 `PC`, 以实现向异常或中断处理程序的转移。这时, `PC` 数据输入端的多路器的选择信号 `selpc` 应为 2, 选择 `base` 地址。这个 `base` 是“硬布线”的地址, 也就是异常或中断处理程序的入口。

CPU 执行 `eret` 指令实现返回。执行这条指令时, CPU 把 `EPC` 的内容写入 `PC`, 因此 `selpc` 应为 1, 以便选择 `epc`。与此同时, `Status` 寄存器的内容要右移 4 位, 即图中的 `sta_right`, 恢复它原来的真面目。

2.4 带有异常和中断处理功能的 CPU Verilog HDL 代码

Verilog HDL 代码与实验 5 的代码有类似的架构, 只是在最顶层加了两个信号: 中断请求 `intr` (输入信号) 和中断确认 `inta` (输出信号)。最顶层代码调用 3 个模块: CPU 模块 (`sccpu_intr`)、指令存储器模块 (`sci_intr`) 和数据存储器模块 (`scd_intr`)。

```
module sc_interrupt (clock, resetn, inst, pc, aluout, memout,
                    mem_clk, intr, inta);
    input clock, resetn, mem_clk, intr;
    output [31:0] inst, pc, aluout, memout;
    output inta;

    wire [31:0] data;
    wire wmem;

    sccpu_intr cpu (clock, resetn, inst, memout, pc, wmem, aluout, data, intr, inta);
    sci_intr i_mem (pc, inst);
    scd_intr d_mem (clock, memout, data, aluout, wmem, mem_clk, mem_clk);
endmodule
```

以下是 CPU 模块的代码。

```

module sccpu_intr(clock, resetn, inst , mem, pc, wmem, alu, data, intr , inta) ;
    input [31:0] inst , mem;
    input clock, resetn , intr;
    output[31:0] pc, alu, data;
    output wmem, inta;
    parameter EXC_BASE = 32'h00000008; // = base = BASE
    wire [31:0] p4,bpc,npc,adr,ra,alua,alub,res,alu_mem;
    wire [3:0] aluc ;
    wire [4:0] reg_dest,wn ;
    wire [1:0] pcsource ;
    wire zero,wmem,wreg,regrt,m2reg,shift,aluimm,jal,sext,overflow;

    wire [31:0] jpc = {p4[31:28] , inst [25:0] , 2'b00} ;
    wire exc , wsta , wcau , wepc , mtc0 ;
    wire [31:0] sta , cau , epc , sta_in , cau_in , epc_in ,
                sta_ll_a0 , epc_ll_a0 , cause , alu_mem_c0 , next_pc;
    wire [1:0] mfc0 , selpc;
    wire [31:0] sa= { 27'b0 , inst[10:6] } ;
    wire [31:0] offset = {imm[13:0] ,inst[15:0] , 2'b00};

    sccu_intr cu (inst[31:26] , inst[25:21] , inst[15:11] , inst[5:0] , zero , wmem ,
                wreg , regrt , m2reg , aluc , shift , aluimm , pcsource , jal , sext ,
                intr , inta , overflow , sta , cause , exc , wsta , wcau , wepc , mtc0 , mfc0 , selpc ) ;
    wire e = sext & inst [ 15 ];
    wire [15:0] imm = {16{e}} ;
    wire [31:0] immediate = {imm, inst[15:0] } ;

    dff32 ip (next_pc, clock , resetn , pc) ;
    cla32 pcplus4(pc , 32'h4 , 1'b0 , p4) ;
    cla32 br_adr(p4, offset , 1'b0 , adr ) ;

    mux2x32 alu_b (data , immediate , aluimm, alub) ;
    mux2x32 alu_a (ra , sa , shift , alua) ;
    mux2x32 result (alu , mem, m2reg , alu_mem) ;
    mux2x32 link (alu_mem_c0, p4 , jal , res) ;
    mux2x5 reg_wn (inst[15:11], inst[20:16], regrt , reg_dest);

    assign wn = reg_dest | {5{jal}};

    mux4x32 nextpc (p4 , adr, ra , jpc , pcsource , npc) ;
    regfile rf(inst[25:21], inst[20:16] ,res , wn , wreg, clock , resetn ,
                ra , data ) ;
    alu_ov al_unit (alua ,alub, aluc , alu ,zero , overflow ) ;

```

CPU 模块新加的与异常或中断有关的模块， 3 个寄存器和 7 个多路器：

```

dffe32 c0_Status (sta_in, clock, resetn, wsta , sta ) ; // Status register
dffe32 c0_Cause (cau_in, clock, resetn, wcau , cau ) ; // cause register
dffe32 c0_EPC (epc_in, clock, resetn, wepc , epc) ; // EPC register

mux2x32 sta_ll (sta_ll_a0, data , mtc0 , sta_in) ; // for Status
mux2x32 sta_l2 ({4'h0, sta[31:4]} , {sta [27:0] , 4'h0} , exc , sta_ll_a0);

mux2x32 cau_ll (cause, data , mtc0 , cau_in); // for Cause
mux2x32 epc_ll (epc_ll_a0, data , mtc0 , epc_in) ;
mux2x32 epc_l2 (pc , npc , inta, epc_ll_a0) ;
mux4x32 irq_pc (npc , epc, EXC_BASE, 32'h0, selpc, next_pc ) ;
mux4x32 fromc0 (alu_mem , sta , cau , epc , mfc0 , alu_mem_c0) ;

endmodule

```

控制部件, 重点是与异常或中断有关的信号:

```

module sccu_intr(op, opl, rd, func, z, wmem, wreg, regrt, m2reg, aluc,
                shift, aluimm, pcsource, jal, sext,
                intr, inta, ov, sta, cause, exc, wsta, wcau, wepc, mtc0, mfc0, selpc);
    input [5:0] op, func;
    input [4:0] opl, rd;
    input z;
    output wreg, regrt, jal, m2reg, shift, aluimm, sext, wmem;
    output [3:0] aluc;
    output [1:0] pcsource;

    input intr, ov;
    input [31:0] sta;
    output inta, exc, wsta, wcau, wepc, mtc0;
    output [1:0] mfc0, selpc;
    output [31:0] cause;

```

区别异常和中断的类型, 4 种: 外部中断、系统调用、未实现的指令以及溢出。

```

wire overflow = ov & ( i_add | i_sub | i_addi );
assign inta = int_int;
wire int_int = sta[0] & intr;
wire exc_sys = sta[1] & i_syscall;
wire exc_uni = sta[2] & unimplemented_inst;
wire exc_ovr = sta[3] & overflow;
assign exc = int_int | exc_sys | exc_uni | exc_ovr;

```

产生 ExcCode: 00: 外部中断、01: 系统调用、10: 未实现的指令、11: 溢出。

```

wire ExcCode0 = i_syscall | overflow;
wire ExcCode1 = unimplemented_inst | overflow;
assign cause = {28'h0, ExcCode1, ExcCode0, 2'b00};

```

产生 3 个寄存器的写使能信号:

```

wire rd_is_status = ( rd == 5'd12 );
wire rd_is_cause = ( rd == 5'd13 );
wire rd_is_epc = ( rd == 5'd14 );

assign mtc0 = i_mtc0;
assign wsta = exc | mtc0 & rd_is_status | i_eret;
assign wcau = exc | mtc0 & rd_is_cause;
assign wepc = exc | mtc0 & rd_is_epc;

```

执行 mfc0 指令时选择寄存器: 00: 原来的、01: Status、10: Cause、11: EPC。

```

assign mfc0[0] = i_mfc0 & rd_is_status | i_mfc0 & rd_is_epc;
assign mfc0[1] = i_mfc0 & rd_is_cause | i_mfc0 & rd_is_epc;

```

PC 的选择: 00: 原来的、01: EPC、10: 异常或中断处理程序的入口。

```

assign selpc[0] = i_eret;
assign selpc[1] = exc;

```

对新加的 4 条指令译码以及确定是否为未实现的指令:

```

wire c0_type = ~op[5] & op[4] & ~op[3] & ~op[2] & ~op[1] & ~op[0];
wire i_mfc0 = c0_type & ~opl[4] & ~opl[3] & ~opl[2] & ~opl[1] & ~opl[0];
wire i_mtc0 = c0_type & ~opl[4] & ~opl[3] & opl[2] & ~opl[1] & ~opl[0];
wire i_eret = c0_type & opl[4] & ~opl[3] & ~opl[2] & ~opl[1] & ~opl[0] &
             ~func[5] & func[4] & func[3] & ~func[2] & ~func[1] & ~func[0];
wire i_syscall = r_type & ~func[5] & ~func[4] & func[3] & func[2] & ~func[1] & ~func[0];

wire unimplemented_inst = ~( i_mfc0 | i_mtc0 | i_eret | i_syscall |
                             i_add | i_sub | i_and | i_or | i_xor | i_sll | i_srl |
                             i_sra | i_jr | i_addi | i_andi | i_ori | i_xori | i_lw |
                             i_sw | i_beq | i_bne | i_lui | i_j | i_jal );

```

以下与实验 5 的内容基本相同, 只是顾及了 mfc0 指令:

```

wire r_type = ~|op;
wire i_add = r_type & func[5] & ~func[4] & ~func[3] & ~func[2] & ~func[1] & ~func[0];
wire i_sub = r_type & func[5] & ~func[4] & ~func[3] & ~func[2] & func[1] & ~func[0];
wire i_and = r_type & func[5] & ~func[4] & ~func[3] & func[2] & ~func[1] & ~func[0];
wire i_or = r_type & func[5] & ~func[4] & ~func[3] & func[2] & ~func[1] & func[0];
wire i_xor = r_type & func[5] & ~func[4] & ~func[3] & func[2] & func[1] & ~func[0];

wire i_sll = r_type & ~func[5] & ~func[4] & ~func[3] & ~func[2] & ~func[1] & ~func[0];
wire i_srl = r_type & ~func[5] & ~func[4] & ~func[3] & ~func[2] & func[1] & ~func[0];
wire i_sra = r_type & ~func[5] & ~func[4] & ~func[3] & ~func[2] & func[1] & func[0];
wire i_jr = r_type & ~func[5] & ~func[4] & func[3] & ~func[2] & ~func[1] & ~func[0];

wire i_addi = ~op[5] & ~op[4] & op[3] & ~op[2] & ~op[1] & ~op[0];
wire i_andi = ~op[5] & ~op[4] & op[3] & op[2] & ~op[1] & ~op[0];
wire i_ori = ~op[5] & ~op[4] & op[3] & op[2] & ~op[1] & op[0];
wire i_xori = ~op[5] & ~op[4] & op[3] & op[2] & op[1] & ~op[0];

wire i_lw = op[5] & ~op[4] & ~op[3] & ~op[2] & op[1] & op[0];
wire i_sw = op[5] & ~op[4] & op[3] & ~op[2] & op[1] & op[0];

wire i_beq = ~op[5] & ~op[4] & ~op[3] & op[2] & ~op[1] & ~op[0];
wire i_bne = ~op[5] & ~op[4] & ~op[3] & op[2] & ~op[1] & op[0];
wire i_lui = ~op[5] & ~op[4] & op[3] & op[2] & op[1] & op[0];
wire i_j = ~op[5] & ~op[4] & ~op[3] & ~op[2] & op[1] & ~op[0];
wire i_jal = ~op[5] & ~op[4] & ~op[3] & ~op[2] & op[1] & op[0];

assign wreg = i_add | i_sub | i_and | i_or | i_xor | i_sll | i_srl | i_sra |
i_andi | i_andi | i_ori | i_xori | i_lw | i_lui | i_jal | i_mfc0;
assign regrt = i_addi | i_andi | i_ori | i_xori | i_lw | i_lui | i_mfc0;
assign jal = i_jal;
assign m2reg = i_lw;
assign shift = i_sll | i_srl | i_sra;
assign aluimm = i_addi | i_andi | i_ori | i_xori | i_lw | i_lui | i_sw;
assign sext = i_addi | i_lw | i_sw | i_beq | i_bne;
assign aluc[3] = i_sra;
assign aluc[2] = i_sub | i_or | i_srl | i_sra | i_ori | i_lui;
assign aluc[1] = i_xor | i_sll | i_srl | i_sra | i_xori | i_beq | i_bne | i_lui;
assign aluc[0] = i_and | i_or | i_sll | i_srl | i_sra | i_andi | i_ori;

assign wmem = i_sw;
assign pcsourse[1] = i_jr | i_j | i_jal;
assign pcsourse[0] = i_beq & z | i_bne & ~z | i_j | i_jal;
endmodule

```

带有溢出标志 v 的 ALU。

```

module alu_ov (a,b,aluc,r, z, v);
input [31:0] a, b;
input [3:0] aluc;
output [31:0] r;
output z, v;

wire [31:0] d_and = a & b;
wire [31:0] d_or = a | b;
wire [31:0] d_xor = a ^ b;
wire [31:0] d_lui = {b[15:0], 16'h0};
wire [31:0] d_and_or = aluc[2]? d_or : d_and;
wire [31:0] d_xor_lui = aluc[2]? d_lui : d_xor;

wire [31:0] d_as,d_sh;

addsub32 as32 (a,b,aluc[2], d_as);
shift shifter (b,a[4:0],aluc[2],aluc[3],d_sh);
mux4x32 select (d_as,d_and_or,d_xor_lui,d_sh, aluc[1:0], r);

assign z = ~|r;

assign v = ~aluc[2] & ~a[31] & ~b[31] & r[31] & ~aluc[1] & ~aluc[0] |
~aluc[2] & a[31] & b[31] & ~r[31] & ~aluc[1] & ~aluc[0] |
aluc[2] & ~a[31] & b[31] & r[31] & ~aluc[1] & ~aluc[0] |
aluc[2] & a[31] & ~b[31] & ~r[31] & ~aluc[1] & ~aluc[0];
endmodule

```

指令存储器:

系统复位 (Reset) 时的入口: 0x00000000。

```

                %      reset:                                %
0: 0800001d; % (00)      j      start                        # entry on reset      %
1: 00000000; % (04)      nop                                #                                %

```

异常或中断处理程序入口: **0x00000008**。查表并跳转。跳转表在数据存储器中。

```

                %      exc_base:                                # exception handler      %
2: 401a6800; % (08)      mfc0 $26, c0_cause                # read cp0 cause reg     %
3: 335b000c; % (0c)      andi $27, $26, 0xc                # get exccode, 2 bits here %
4: 8f7b0020; % (10)      lw  $27, j_table($27)             # get address from table  %
5: 00000000; % (14)      nop                                #                                %
6: 03600008; % (18)      jr  $27                           # jump to that address   %
7: 00000000; % (1c)      nop                                #                                %

```

外部中断处理程序入口: **0x00000030**。目前是直接返回。

```

                %      int_entry:                                # 0. interrupt handler   %
c: 00000000; % (30)      nop                                # deal with interrupt here %
d: 42000018; % (34)      eret                               # return from interrupt   %
e: 00000000; % (38)      nop                                #                                %

```

系统调用指令的入口: **0x0000003c**。EPC 加 4 后就返回了。

```

                %      sys_entry:                                # 1. syscall handler     %
f: 00000000; % (3c)      nop                                # do something here       %
                %      epc_plus4:                                #                                %
10: 401a7000; % (40)      mfc0 $26, c0_epc                 # get epc                 %
11: 235a0004; % (44)      addi $26, $26, 4                 # epc + 4                 %
12: 409a7000; % (48)      mtc0 $26, c0_epc                 # epc <- epc + 4         %
13: 42000018; % (4c)      eret                               # return from exception   %
14: 00000000; % (50)      nop                                #                                %

```

未实现的指令的入口: **0x00000054**。EPC 加 4 后就返回了。

```

                %      uni_entry:                                # 2. unimpl. inst. handler %
15: 00000000; % (54)      nop                                # do something here       %
16: 08000010; % (58)      j      epc_plus4                 # return                  %
17: 00000000; % (5c)      nop                                #                                %

```

溢出处理程序的入口: **0x00000068**。EPC 加 4 后就返回了。

```

                %      ovf_entry:                                # 3. overflow handler     %
1a: 00000000; % (68)      nop                                # do something here       %
1b: 08000010; % (6c)      j      epc_plus4                 # return                  %
1c: 00000000; % (70)      nop                                #                                %

```

开中断:

```

%      start:                                     #                                     %
1d: 2008000f; % (74)      addi $8, $0, 0xf          # im[3:0] <- 1111          %
1e: 40886000; % (78)      mtc0 $8, c0_status        # exc/intr enable         %

```

测试溢出:

```

1f: 8c080048; % (7c)      lw      $8, 0x48($0)      # try overflow exception  %
20: 8c09004c; % (80)      lw      $9, 0x4c($0)      # caused by add           %
%      ov:                                           #                                     %
21: 01094020; % (84)      add     $9, $9, $8         # overflow                %
22: 00000000; % (88)      nop                                     #                                     %

```

测试系统调用:

```

%      sys:                                           #                                     %
23: 0000000c; % (8c)      syscall                    # system call             %
24: 00000000; % (90)      nop                          #                                     %

```

测试未实现的指令:

```

%      unimpl:                                       #                                     %
25: 0128001a; % (94)      div     $9, $8             # div, but not implemented %
26: 00000000; % (98)      nop                          #                                     %

```

测试外部中断: 在此期间由外部送来中断信号 **intr**。

```

%      int:                                           #                                     %
27: 34040050; % (9c)      ori     $4, $1, 0x50       # address of data[0]      %
28: 20050004; % (a0)      addi    $5, $0, 4           # counter                 %
29: 00004020; % (a4)      add     $8, $0, $0          # sum <- 0                %
%      loop:                                           #                                     %
2a: 8c890000; % (a8)      lw      $9, 0($4)          # load data               %
2b: 20840004; % (ac)      addi    $4, $4, 4           # address + 4             %
2c: 01094020; % (b0)      add     $8, $8, $9          # sum                     %
2d: 20a5ffff; % (b4)      addi    $5, $5, -1         # counter - 1             %
2e: 14a0ffff; % (b8)      bne     $5, $0, loop        # finish?                 %
2f: 00000000; % (bc)      nop                                     #                                     %
%      finish:                                         #                                     %
30: 08000030; % (c0)      j       finish              # dead loop               %

```

数据存储器: 0 到 1f = 00000000

跳转表:

```

% address table for internal exceptions and external interrupt %
8 : 00000030;      % (20) int_entry # 0. address for interrupt %
9 : 0000003c;      % (24) sys_entry # 1. address for system call %
a : 00000054;      % (28) uni_entry # 2. address for unimpl. instruction %

```

为测试溢出而准备的数据:

```
b : 00000068;      % (2c) ovf_entry # 3. address for signed overflow      %
12 : 00000002;      % (48) for testing overflow                          %
13 : 7fffffff;      % (4c) 2 + max_int -> overflow                      %
14 : 000000a3;      % (50) data[0]  0 +  a3 =  a3                        %
15 : 00000027;      % (54) data[1]  a3 +  27 =  ca                       %
16 : 00000079;      % (58) data[2]  ca +   79 = 143                      %
17 : 00000115;      % (5c) data[3] 143 + 115 = 258                      %
```

3 CPU 的异常与中断测试

复位时 CPU 从 0x00000000 地址开始执行。因为 0x00000000 地址的指令是 j 指令, 跳转到 0x00000074 去执行。

```
0 pc=00000000 inst=0800001d aluout=00000000 memout=xxxxxxxx
5 pc=00000000 inst=0800001d aluout=00000000 memout=00000000
```

开中断后, 在 PC = 0x00000084 处的 add 指令产生溢出, 从而进入异常或中断处理程序的总入口 0x00000008。

```
20 pc=00000074 inst=2008000f aluout=0000000f memout=00000000
40 pc=00000078 inst=40886000 aluout=0000000f memout=00000000
60 pc=0000007c inst=8c080048 aluout=00000048 memout=00000000
65 pc=0000007c inst=8c080048 aluout=00000048 memout=00000002
80 pc=00000080 inst=8c09004c aluout=0000004c memout=00000002
85 pc=00000080 inst=8c09004c aluout=0000004c memout=7fffffff
100 pc=00000084 inst=01094020 aluout=80000001 memout=7fffffff
105 pc=00000084 inst=01094020 aluout=80000001 memout=00000000
120 pc=00000008 inst=401a6800 aluout=00000000 memout=00000000
```

是执行以下 7 条指令的仿真输出。

```
00: 0800001d; % (00) j start # entry on reset %
1d: 2008000f; % (74) addi $8, $0, 0xf # im[3:0] <- 1111 %
1e: 40886000; % (78) mtc0 $8, c0_status # exc/intr enable %
1f: 8c080048; % (7c) lw $8, 0x48($0) # try overflow exception %
20: 8c09004c; % (80) lw $9, 0x4c($0) # caused by add %
21: 01094020; % (84) add $9, $9, $8 # overflow %
02: 401a6800; % (08) mfc0 $26, c0_cause # read cp0 cause reg %
```

检查异常类型并转入相应的处理程序的入口处。由于本次的异常是溢出, 因此转入 0c00000068。然后从 0x0000006c 转去做 EPC 加 4。

```
140 pc=0000000c inst=335b000c aluout=0000000c memout=00000000
160 pc=00000010 inst=8f7b0020 aluout=0000002c memout=00000000
165 pc=00000010 inst=8f7b0020 aluout=0000002c memout=00000068
180 pc=00000014 inst=00000000 aluout=00000000 memout=00000068
185 pc=00000014 inst=00000000 aluout=00000000 memout=00000000
200 pc=00000018 inst=03600008 aluout=00000068 memout=00000000
220 pc=00000068 inst=00000000 aluout=00000000 memout=00000000
240 pc=0000006c inst=08000010 aluout=00000000 memout=00000000
```

是执行以下 6 条指令的输出。

```
03: 335b000c; % (0c) andi $27, $26, 0xc # get exccode, 2 bits here %
04: 8f7b0020; % (10) lw $27, j_table($27) # get address from table %
05: 00000000; % (14) nop # %
06: 03600008; % (18) jr $27 # jump to that address %
1a: 00000000; % (68) nop # do something here %
1b: 08000010; % (6c) j epc_plus4 # return %
```

完成 EPC 加 4, 然后返回。返回后执行系统调用 `syscall` 指令, 又进入异常或中断处理程序的总入口处。

```
260 pc=00000040 inst=401a7000 aluout=0000000c memout=00000000
280 pc=00000044 inst=235a0004 aluout=00000088 memout=00000000
300 pc=00000048 inst=409a7000 aluout=00000088 memout=00000000
320 pc=0000004c inst=42000018 aluout=00000000 memout=00000000
340 pc=00000088 inst=00000000 aluout=00000000 memout=00000000
360 pc=0000008c inst=0000000c aluout=00000000 memout=00000000
380 pc=00000008 inst=401a6800 aluout=00000088 memout=00000000
400 pc=0000000c inst=335b000c aluout=00000004 memout=00000000
```

是执行以下 8 条指令的输出。

```
10: 401a7000; % (40) mfc0 $26, c0_epc      # get epc %
11: 235a0004; % (44) addi $26, $26, 4      # epc + 4 %
12: 409a7000; % (48) mtc0 $26, c0_epc      # epc <- epc + 4 %
13: 42000018; % (4c) eret                  # return from exception %
22: 00000000; % (88) nop                   # %
23: 0000000c; % (8c) syscall                # system call %
02: 401a6800; % (08) mfc0 $26, c0_cause    # read cp0 cause reg %
03: 335b000c; % (0c) andi $27, $26, 0xc    # get exccode, 2 bits here %
```

进入系统调用异常的处理程序。

```
400 pc=0000000c inst=335b000c aluout=00000004 memout=00000000
420 pc=00000010 inst=8f7b0020 aluout=00000024 memout=00000000
425 pc=00000010 inst=8f7b0020 aluout=00000024 memout=0000003c
440 pc=00000014 inst=00000000 aluout=00000000 memout=0000003c
445 pc=00000014 inst=00000000 aluout=00000000 memout=00000000
460 pc=00000018 inst=03600008 aluout=0000003c memout=00000000
480 pc=0000003c inst=00000000 aluout=00000000 memout=00000000
500 pc=00000040 inst=401a7000 aluout=00000004 memout=00000000
520 pc=00000044 inst=235a0004 aluout=00000090 memout=00000000
540 pc=00000048 inst=409a7000 aluout=00000090 memout=00000000
```

它是执行以下 7 条指令的输出。

```
04: 8f7b0020; % (10) lw $27, j_table($27) # get address from table %
05: 00000000; % (14) nop                  # %
06: 03600008; % (18) jr $27               # jump to that address %
0f: 00000000; % (3c) nop                  # do something here %
10: 401a7000; % (40) mfc0 $26, c0_epc      # get epc %
11: 235a0004; % (44) addi $26, $26, 4      # epc + 4 %
12: 409a7000; % (48) mtc0 $26, c0_epc      # epc <- epc + 4 %
```

仿真执行 `div r9, r8`。这是一条合法的 MIPS 的整数除法指令, 但目前的 CPU 没有实现它, 因此产生未实现的指令异常。也是进入异常或中断处理程序的总入口。

```
560 pc=0000004c inst=42000018 aluout=00000000 memout=00000000
580 pc=00000090 inst=00000000 aluout=00000000 memout=00000000
600 pc=00000094 inst=0128001a aluout=00000000 memout=00000000
620 pc=00000008 inst=401a6800 aluout=00000090 memout=00000000
640 pc=0000000c inst=335b000c aluout=00000008 memout=00000000
660 pc=00000010 inst=8f7b0020 aluout=00000028 memout=00000000
665 pc=00000010 inst=8f7b0020 aluout=00000028 memout=00000054
680 pc=00000014 inst=00000000 aluout=00000000 memout=00000054
685 pc=00000014 inst=00000000 aluout=00000000 memout=00000000
```

它是执行以下 7 条指令的输出。

```
13: 42000018; % (4c) eret          # return from exception %
24: 00000000; % (90) nop           # %
25: 0128001a; % (94) div $9, $8    # div, but not implemented %
02: 401a6800; % (08) mfc0 $26, c0_cause # read cp0 cause reg %
03: 335b000c; % (0c) andi $27, $26, 0xc # get exccode, 2 bits here %
04: 8f7b0020; % (10) lw $27, j_table($27) # get address from table %
05: 00000000; % (14) nop           # %
```

仿真进入未实现的指令异常的处理程序。

```
700 pc=00000018 inst=03600008 aluout=00000054 memout=00000000
705 pc=00000018 inst=03600008 aluout=00000054 memout=00000027
720 pc=00000054 inst=00000000 aluout=00000000 memout=00000027
725 pc=00000054 inst=00000000 aluout=00000000 memout=00000000
740 pc=00000058 inst=08000010 aluout=00000000 memout=00000000
760 pc=00000040 inst=401a7000 aluout=00000008 memout=00000000
780 pc=00000044 inst=235a0004 aluout=00000098 memout=00000000
800 pc=00000048 inst=409a7000 aluout=00000098 memout=00000000
820 pc=0000004c inst=42000018 aluout=00000000 memout=00000000
```

它是执行以下 7 条指令的输出。

```
06: 03600008; % (18) jr $27        # jump to that address %
15: 00000000; % (54) nop           # do something here %
16: 08000010; % (58) j epc_plus4   # return %
10: 401a7000; % (40) mfc0 $26, c0_epc # get epc %
11: 235a0004; % (44) addi $26, $26, 4 # epc + 4 %
12: 409a7000; % (48) mtc0 $26, c0_epc # epc <- epc + 4 %
13: 42000018; % (4c) eret          # return from exception %
```

外部中断是异步的,在仿真中通过置 **intr** 信号为 1 来产生外部中断。从仿真可以看出,中断出现在 CPU 正在执行 PC = 0x000000AC 处的指令。CPU 执行完该指令后,转去 0x00000008。

```
840 pc=00000098 inst=00000000 aluout=00000000 memout=00000000
860 pc=0000009c inst=34040050 aluout=00000050 memout=00000000
865 pc=0000009c inst=34040050 aluout=00000050 memout=000000a3
880 pc=000000a0 inst=20050004 aluout=00000004 memout=000000a3
885 pc=000000a0 inst=20050004 aluout=00000004 memout=00000000
900 pc=000000a4 inst=00004020 aluout=00000000 memout=00000000
920 pc=000000a8 inst=8c890000 aluout=00000050 memout=00000000
925 pc=000000a8 inst=8c890000 aluout=00000050 memout=000000a3
940 pc=000000ac inst=20840004 aluout=00000054 memout=000000a3
945 pc=000000ac inst=20840004 aluout=00000054 memout=00000027
960 pc=00000008 inst=401a6800 aluout=00000098 memout=00000027
965 pc=00000008 inst=401a6800 aluout=00000098 memout=00000000
```

它是执行以下 7 条指令的输出。

```
26: 00000000; % (98) nop           # %
27: 34040050; % (9c) ori $4, $1, 0x50 # address of data[0] %
28: 20050004; % (a0) addi $5, $0, 4   # counter %
29: 00004020; % (a4) add $8, $0, $0    # sum <- 0 %
2a: 8c890000; % (a8) lw $9, 0($4)     # load data %
2b: 20840004; % (ac) addi $4, $4, 4   # address + 4 %
02: 401a6800; % (08) mfc0 $26, c0_cause # read cp0 cause reg %
```

从中断处理程序返回到 PC = 0x000000 B0

```
980 pc=0000000c inst=335b000c aluout=00000000 memout=00000000
1000 pc=00000010 inst=8f7b0020 aluout=00000020 memout=00000000
1005 pc=00000010 inst=8f7b0020 aluout=00000020 memout=00000030
1020 pc=00000014 inst=00000000 aluout=00000000 memout=00000030
1025 pc=00000014 inst=00000000 aluout=00000000 memout=00000000
1040 pc=00000018 inst=03600008 aluout=00000030 memout=00000000
1060 pc=00000030 inst=00000000 aluout=00000000 memout=00000000
1080 pc=00000034 inst=42000018 aluout=00000000 memout=00000000
1100 pc=000000b0 inst=01094020 aluout=000000a3 memout=00000000
1105 pc=000000b0 inst=01094020 aluout=000000a3 memout=00000030
```

它是执行以下 7 条指令的输出。

```
03: 335b000c; % (0c) andi $27, $26, 0xc # get exccode, 2 bits here %
04: 8f7b0020; % (10) lw $27, j_table($27) # get address from table %
05: 00000000; % (14) nop # %
06: 03600008; % (18) jr $27 # jump to that address %
0c: 00000000; % (30) nop # deal with interrupt here %
0d: 42000018; % (34) eret # return from interrupt %
2c: 01094020; % (b0) add $8, $8, $9 # sum %
```

后面是一些指令的循环执行。

```
1120 pc=000000b4 inst=20a5ffff aluout=00000003 memout=00000030
1125 pc=000000b4 inst=20a5ffff aluout=00000003 memout=00000000
1140 pc=000000b8 inst=14a0ffffb aluout=00000003 memout=00000000
1160 pc=000000a8 inst=8c890000 aluout=00000054 memout=00000000
1165 pc=000000a8 inst=8c890000 aluout=00000054 memout=00000027
1180 pc=000000ac inst=20840004 aluout=00000058 memout=00000027
1185 pc=000000ac inst=20840004 aluout=00000058 memout=00000079
1200 pc=000000b0 inst=01094020 aluout=000000ca memout=00000079
1205 pc=000000b0 inst=01094020 aluout=000000ca memout=00000002
1220 pc=000000b4 inst=20a5ffff aluout=00000002 memout=00000002
1225 pc=000000b4 inst=20a5ffff aluout=00000002 memout=00000000
1240 pc=000000b8 inst=14a0ffffb aluout=00000002 memout=00000000
1260 pc=000000a8 inst=8c890000 aluout=00000058 memout=00000000
1265 pc=000000a8 inst=8c890000 aluout=00000058 memout=00000079
1280 pc=000000ac inst=20840004 aluout=0000005c memout=00000079
1285 pc=000000ac inst=20840004 aluout=0000005c memout=00000115
1300 pc=000000b0 inst=01094020 aluout=00000143 memout=00000115
1305 pc=000000b0 inst=01094020 aluout=00000143 memout=00000000
1320 pc=000000b4 inst=20a5ffff aluout=00000001 memout=00000000
1340 pc=000000b8 inst=14a0ffffb aluout=00000001 memout=00000000
1360 pc=000000a8 inst=8c890000 aluout=0000005c memout=00000000
1365 pc=000000a8 inst=8c890000 aluout=0000005c memout=00000115
1380 pc=000000ac inst=20840004 aluout=00000060 memout=00000115
1385 pc=000000ac inst=20840004 aluout=00000060 memout=00000000
1400 pc=000000b0 inst=01094020 aluout=00000258 memout=00000000
1405 pc=000000b0 inst=01094020 aluout=00000258 memout=00000079
1420 pc=000000b4 inst=20a5ffff aluout=00000000 memout=00000079
1425 pc=000000b4 inst=20a5ffff aluout=00000000 memout=00000000
1440 pc=000000b8 inst=14a0ffffb aluout=00000000 memout=00000000
1460 pc=000000bc inst=00000000 aluout=00000000 memout=00000000
1480 pc=000000c0 inst=08000030 aluout=00000000 memout=00000000
```